TEK-UP
Private Higher School of Technologies & Engineering

Projet DevOps

GLSI2 - 2025/2026

HealthGuardian AI

Plateforme intelligente de prévention des maladies chroniques

Réalisé par:

Imen sahli - Hanin sahli - Amira sarray- Ons nacer

Encadrépar:

MmeSiwarZrelli

Janvier 2026

Chapitre1

Introduction

1.1 Présentation du projet:

Health guardian AI Plateforme intelligente de prévention des maladies chroniques développée avec une architecture microservices:

— Backend SpringBoot(Java)

— Frontend React JS

— Service IA Python

— Base de données MySQL

1.2 Objectifs DevOps:

— Automatisation complète du pipeline CI/CD

— Conteneurisation avec Docker

— Tests automatisés et qualité du code

— Déploiement automatique sur DockerHub

Chapitre2

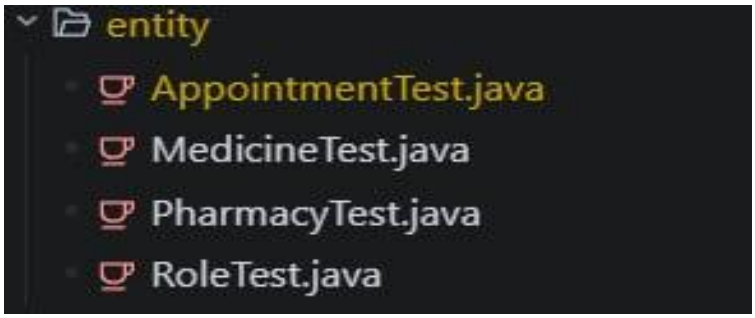
Tests Locaux

2.1 Tests Exécutés Localement

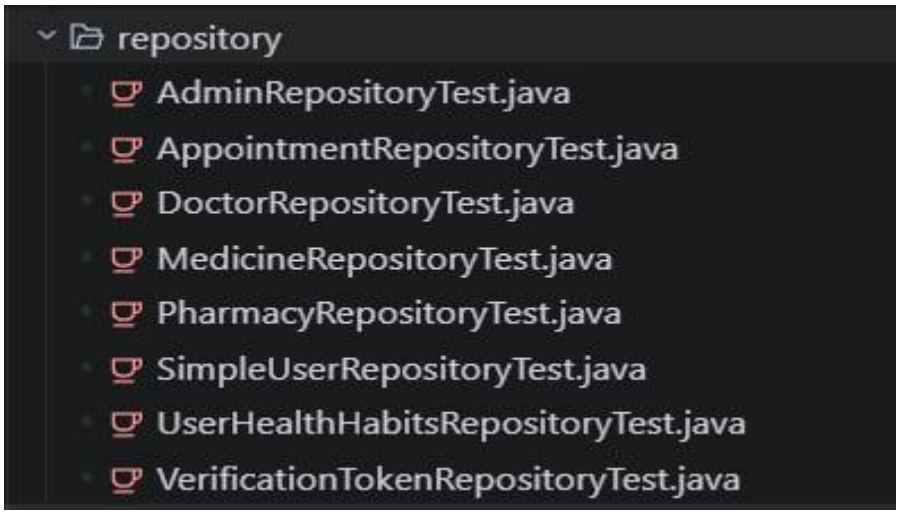
```
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 220, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jacoco:0.8.11:report (report) @ healthguardian-backend ---
[INFO] Loading execution data file C:\Users\HaninSahli\Desktop\healthguardianai-backend\healthguardia
n\target\jacoco.exec
[INFO] Analyzed bundle 'HealthGuardian AI Backend' with 24 classes
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  57.247 s
[INFO] Finished at: 2026-01-04T23:06:07+01:00
[INFO] ------------------------------------------------------------------------
```

2.2 Test par module




entity
AppointmentTest.java
MedicineTest.java
PharmacyTest.java
RoleTest.java


dto
AuthResponseTest.java
SignInRequestTest.java

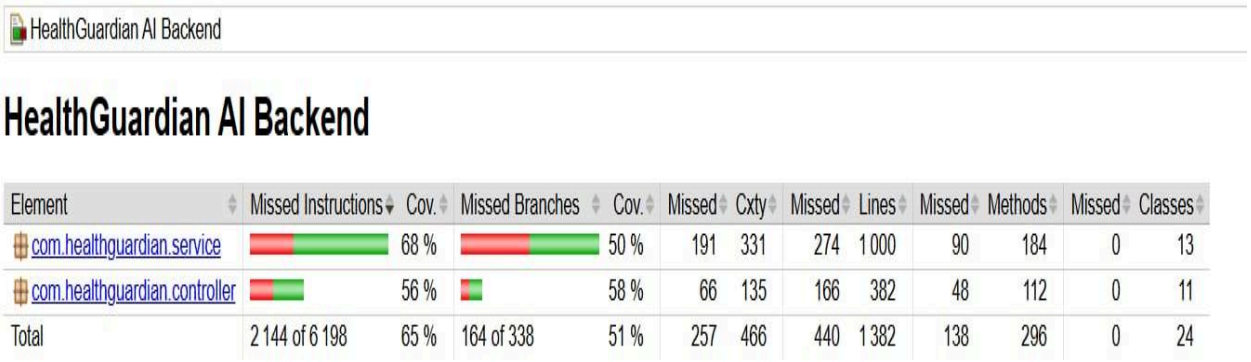

repository
AdminRepositoryTest.java
AppointmentRepositoryTest.java
DoctorRepositoryTest.java
MedicineRepositoryTest.java
PharmacyRepositoryTest.java
SimpleUserRepositoryTest.java
UserHealthHabitsRepositoryTest.java
VerificationTokenRepositoryTest.java


service
AdminServiceTest.java
AppointmentServiceTest.java
AuthServiceTest.java
DoctorAvailabilityServiceTest.java
DoctorPatientServiceTest.java
DoctorServiceTest.java
JwtServiceTest.java
MedicineServiceTest.java
PharmacyServiceTest.java
UserHealthHabitsServiceTest.java
UserServiceTest.java

2.3 analyse de couverture de code locale

2.3.1 rapport jacoco

Après avoir exécuté les tests unitaires et d'intégration localement, nous avons généré le rapport de couverture de code avec JaCoCo pour analyser la qualité de nos tests.



Raffinement très court – Tests automatisés

Une stratégie de tests automatisés complète a été mise en place selon la pyramide des tests, garantissant la fiabilité et la sécurité de l'application HealthGuardian.

La majorité des tests sont unitaires pour valider la logique métier, complétés par des tests fonctionnels pour l'API REST et des tests d'intégration pour la couche données.

Au total, 220 tests ont été exécutés avec un taux de réussite de 100 %, en utilisant JUnit 5, Mockito, Spring Boot Test et H2.

Cette approche assure une application robuste, sécurisée et prête pour l'intégration continue (CI/CD).

2.4 Implémentation des relations et des formulaires (DTO)

L'architecture de HealthGuardian s'appuie sur JPA/Hibernate pour gérer les relations entre entités.

Afin de sécuriser et simplifier les échanges via l'API, le patron DTO (Data Transfer Object) est utilisé comme un formulaire intermédiaire, permettant de valider les données et de découpler la base de données de la couche API.

1. Relation Many-to-One : Rendez-vous – Patient & Médecin

Chaque rendez-vous est associé à un patient et un médecin uniques.

L'entité Appointment définit cette relation via @ManyToOne, tandis que le DTO AppointmentCreateDTO permet au client d'envoyer uniquement les identifiants nécessaires, avec des règles de validation (champs obligatoires, date future).

2. Relation One-to-Many : Pharmacie & Médicaments

Une pharmacie peut gérer plusieurs médicaments.

La relation est configurée avec @OneToMany et un cascade delete, garantissant la suppression automatique des médicaments associés. La création des médicaments se fait via un endpoint lié à l'ID de la pharmacie, assurant une association cohérente.

3. Héritage : Gestion des utilisateurs

Le système gère plusieurs types d'utilisateurs (patients, médecins, administrateurs) via une stratégie d'héritage JOINED, centralisant les données communes et séparant les spécificités.

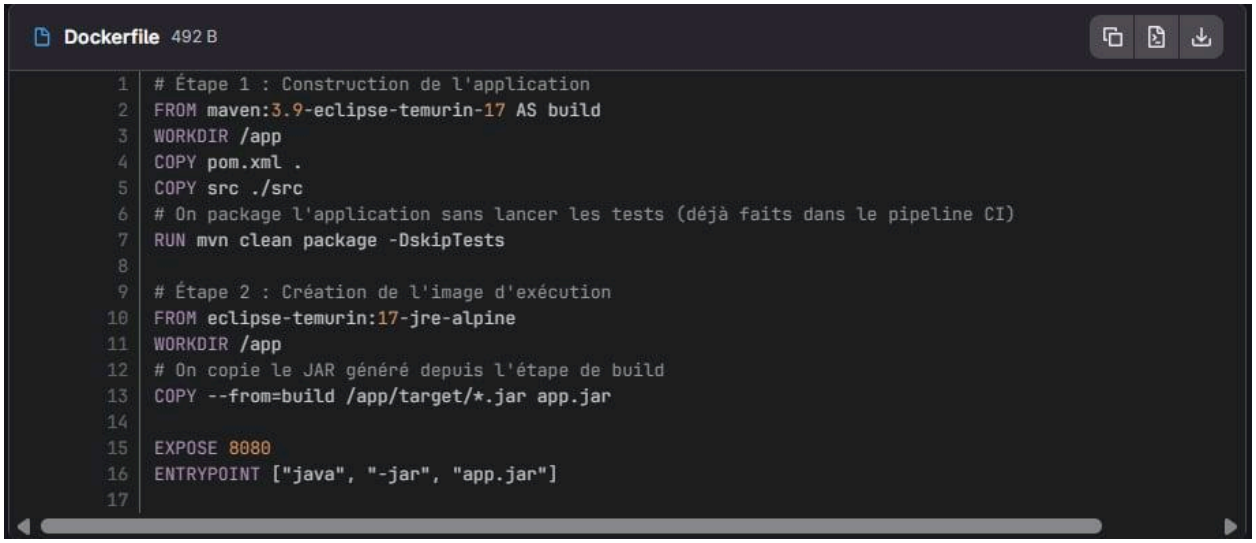
Un formulaire d'inscription unique (DTO) permet de créer dynamiquement le bon type d'utilisateur selon le rôle sélectionné.

Chapitre 3

Dockerisation et built Local

3.1 Dockerfiles Optimisés

3.1.1 Dockerfile Backend



Caractéristiques:

—Build multi-étapes : compilation Maven (Java 17) puis image finale légère.

—Tests ignorés lors du build (-DskipTests) pour accélérer la CI.

—Runtime optimisé avec eclipse-temurin:17-jre-alpine.

—Application Java exposée sur le port 8080, lancée via un JAR unique.

3.1.2 docker file frontend

Dockerfile 348 B

```
# Étape de construction
FROM node:20-alpine as build-stage
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

# Étape de production
FROM nginx:alpine as production-stage
COPY --from=build-stage /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]

```

Caractéristiques :

—Build multi-étapes : build de l'application frontend avec Node.js 20 (Alpine).

—Dépendances installées via npm install, puis génération des fichiers statiques avec npm run build.

—Image finale légère basée sur Nginx Alpine pour l'environnement de production.

—Application statique exposée sur le port 80 avec une configuration Nginx personnalisée.

3.1.3 Docker file ai service

Dockerfile 579 B

```
# Utiliser une image Python officielle légère
FROM python:3.11-slim

# Définir le répertoire de travail dans le conteneur
WORKDIR /app

# Copier le fichier des dépendances
COPY requirements.txt .

# Installer les dépendances
# --no-cache-dir permet de réduire la taille de l'image
RUN pip install --no-cache-dir -r requirements.txt

# Copier le reste du code de l'application
COPY . .

# Exposer le port sur lequel l'application s'exécute
EXPOSE 8000

# Commande pour démarrer l'application avec Uvicorn
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]

```

Caractéristiques :

—Image de base légère : Python 3.11 slim pour réduire la taille du conteneur.

—Installation optimisée des dépendances via pip --no-cache-dir.

—Application Python copiée dans /app et exécutée avec Uvicorn.

—API exposée sur le port 8000, accessible via 0.0.0.0 (toutes les interfaces réseau).

3.2 Docker Compose pour orchestration

docker-compose.yml 3.54 KiB

```yaml
version: '3.8'

services:
  # ========================================
  # 1. BASE DE DONNÉES (MySQL)
  # ========================================
  db:
    image: mysql:8.0
    container_name: healthguardian-db
    restart: always
    environment:
      MYSQL_DATABASE: healthguardian_db
      MYSQL_ROOT_PASSWORD: rootpassword  # Mot de passe sécurisé pour la prod
    ports:
      - "3306:3306"
    volumes:
      - db_data:/var/lib/mysql
    command: --default-authentication-plugin=mysql_native_password

  # ========================================
  # 2. PHPMYADMIN (Interface DB)
  # ========================================
  phpmyadmin:
    image: phpmyadmin/phpmyadmin
    container_name: healthguardian-phpmyadmin
    restart: always
    depends_on:
      - db
    environment:
      PMA_HOST: db
      PMA_PORT: 3306
    ports:
      - "8082:80" # Accessible via http://localhost:8082

```

```yaml
  # ========================================
  # 3. BACKEND (Spring Boot)
  # ========================================
  backend:
    image: sahlihanin/healthguardian-backend:latest
    container_name: healthguardian-backend
    restart: on-failure
    depends_on:
      - db
    environment:
      # Configuration SGBD
      SPRING_DATASOURCE_URL: jdbc:mysql://db:3306/healthguardian_db?useSSL=false&allowPublicKeyRetrieva
      SPRING_DATASOURCE_USERNAME: root
      SPRING_DATASOURCE_PASSWORD: rootpassword
      SPRING_DATASOURCE_DRIVER_CLASS_NAME: com.mysql.cj.jdbc.Driver

      # Configuration JPA (Update pour ne pas perdre les données à chaque redémarrage)
      SPRING_JPA_HIBERNATE_DDL_AUTO: update

      # Configuration Serveur
      SERVER_PORT: 8081

      # Connexion avec le service IA
      AI_SERVICE_URL: http://ai-service:8000

      # CORS (Autoriser le frontend Dockerisé)
      CORS_ALLOWED_ORIGINS: http://localhost:3000
    ports:
      - "8081:8081"

```

```yaml
  # ========================================
  # 4. SERVICE IA (Python/FastAPI/Uvicorn)
  # ========================================
  ai-service:
    image: sahlihanin/healthguardian-ai:latest
    container_name: healthguardian-ai
    restart: on-failure
    ports:
      - "8000:8000"
    environment:
      # Si votre IA a besoin de se connecter à la DB ou au Backend, ajoutez les variables ici
      DB_HOST: db
      DB_USER: root
      DB_PASSWORD: rootpassword

  # ========================================
  # 5. FRONTEND (React/Nginx)
  # ========================================
  frontend:
    image: sahlihanin/healthguardian-frontend:latest
    container_name: healthguardian-frontend
    restart: always
    depends_on:
      - backend
    ports:
      - "3000:80" # Mappé sur 3000 pour correspondre à vos habitudes (http://localhost:3000)
    environment:
      # Attention : Cette variable doit souvent être "baker" dans l'image au moment du build
      # ou injectée via un script au démarrage du conteneur Nginx si configuré.
      REACT_APP_API_URL: http://localhost:8081/api

```

```yaml
  # ==========================================
  # 6. SONARQUBE (Code Quality)
  # ==========================================
  sonarqube:
    image: sonarqube:community
    container_name: healthguardian-sonarqube
    restart: always
    ports:
      - "9000:9000"
    environment:
      - SONAR_ES_BOOTSTRAP_CHECKS_DISABLE=true
    volumes:
      - sonarqube_data:/opt/sonarqube/data
      - sonarqube_extensions:/opt/sonarqube/extensions
      - sonarqube_logs:/opt/sonarqube/logs

volumes:
  db_data:
  sonarqube_data:
  sonarqube_extensions:
  sonarqube_logs:

```

Explication de la configuration :

- Architecture multi-services : base de données MySQL, backend Spring Boot, service IA FastAPI, frontend React, outils phpMyAdmin et SonarQube.
- Communication interne Docker via noms de services (db, backend, ai-service) sans exposition inutile.
- Persistance des données assurée par des volumes Docker (MySQL et SonarQube).
- Ports exposés pour l'accès utilisateur : frontend (3000), backend (8081), phpMyAdmin (8082) et SonarQube (9000).

3.3 Build et test local avec Docker compose

```
PS C:\Users\HaninSahli\desktop\HealthGuardian-Deploy> docker-compose up -d
time="2026-01-04T19:38:40+01:00" level=warning msg="C:\\Users\\HaninSahli\\desktop\\HealthGuardian-Deploy\\docker-compos
e.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 5/5
 ✔Container healthguardian-db          Running                                                              0.0s
 ✔Container healthguardian-phpmyadmin  Running                                                              0.0s
 ✔Container healthguardian-ai          Started                                                              0.0s
 ✔Container healthguardian-backend     Started                                                              0.3s
 ✔Container healthguardian-frontend    Started                                                              0.3s
PS C:\Users\HaninSahli\desktop\HealthGuardian-Deploy>
```

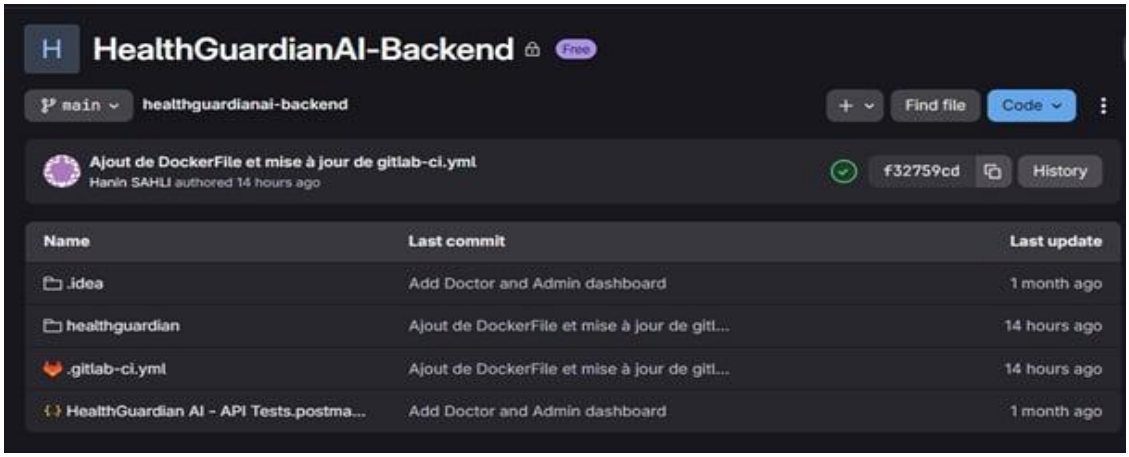

healthguardian- -
- -
0.89% 29 seconds ago
healthguardia 0bd3b66b8008
sahlihanin/h 3000:80
0%
2 minutes ago
healthguardia 05d54c588b66
sahlihanin/h 8081:8081
0.14%
2 minutes ago
healthguardia d4a82af13a0c
phpmyadmi 8082:80
0.02%
5 minutes ago
healthguardia 481608c2a06f
mysql:8.0 3306:3306
0.73%
5 minutes ago

Chapitre 4

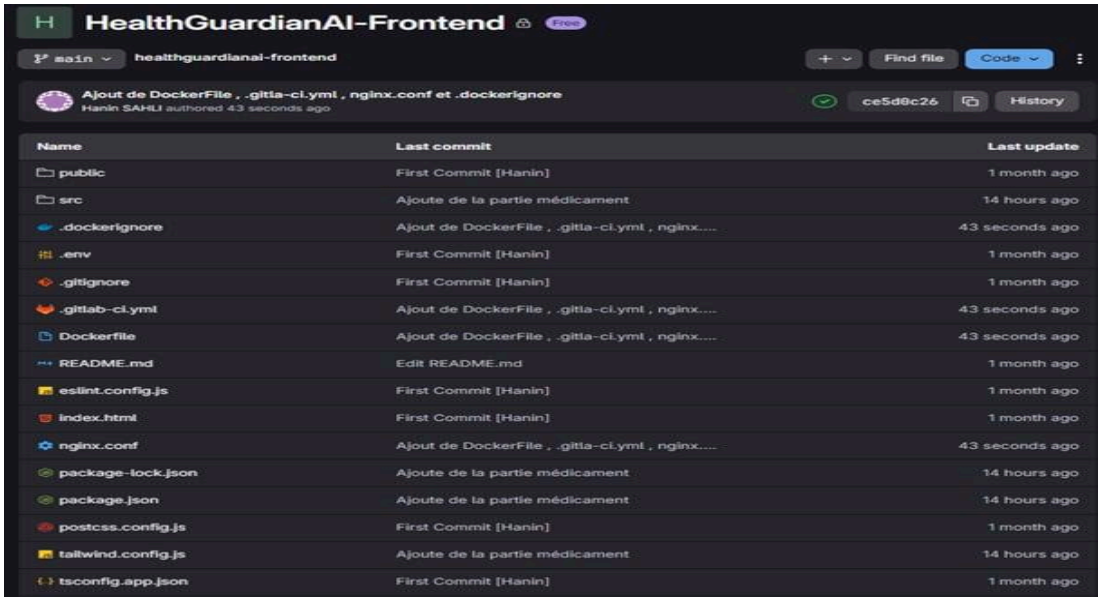
Configuration gitlab ci cd

4.1 Set up des dépôts du projet

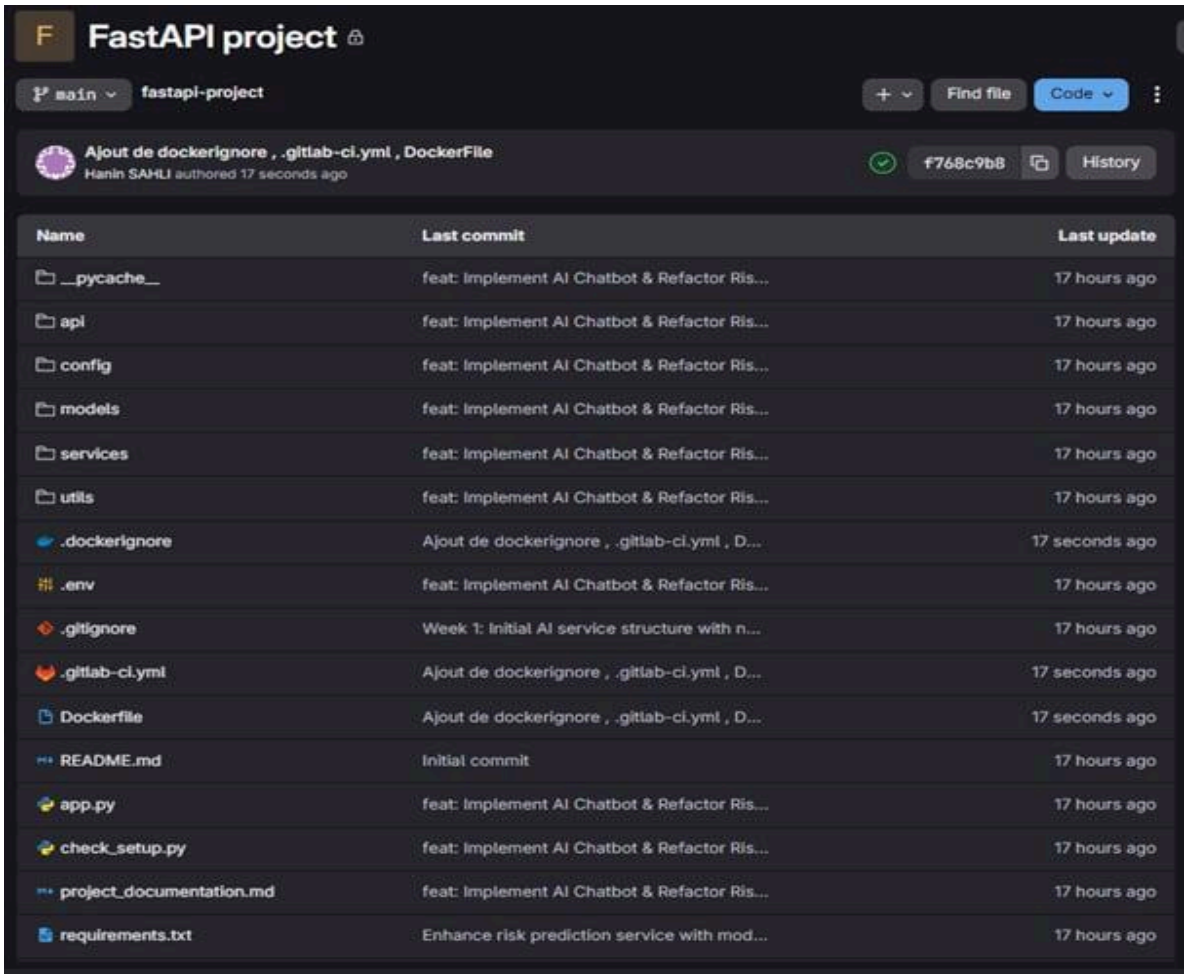
4.1.1 creation de depot backend



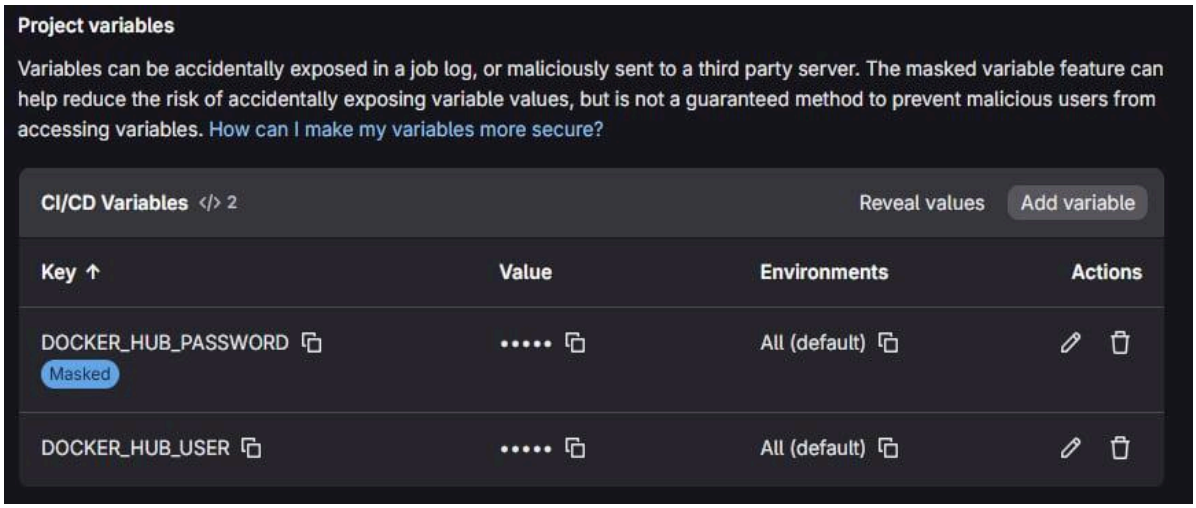
4.1.2 creation de depot frontend



4.1.3 creation de depot AI



4.1.4 creation des variables de projet



4.2 Fichier .gitlab-ci.yml / Backend

.gitlab-ci.yml 2.75 KiB

```yaml
variables:
  MAVEN_OPTS: "-Dmaven.repo.local=$CI_PROJECT_DIR/.m2/repository"

cache:
  paths:
    - .m2/repository/

stages:
  - test
  - analysis
  - deploy

# ================================================================
# STAGE 1: TESTS
# ================================================================

tests:
  stage: test
  image: maven:3.9-eclipse-temurin-17
  before_script:
    - cd healthguardian
  script:
    - echo " Running 39 tests (unit + integration + functional)..."
    - mvn clean test
  artifacts:
    when: always
    reports:
      junit: target/surefire-reports/TEST-*.xml
    paths:
      - target/surefire-reports/
      - target/site/jacoco/
  only:
    - main
    - merge_requests

# ================================================================
# STAGE 2: ANALYSE DE CODE
# ================================================================

code-coverage:
  stage: analysis
  image: maven:3.9-eclipse-temurin-17
  script:
    - cd healthguardian
    - echo " Generating code coverage report with JaCoCo..."
    - mvn jacoco:report
  artifacts:
    paths:
      - target/site/jacoco/
  coverage: '/Total.*?([0-9]{1,3})%/'
  only:
    - main
    - merge_requests

code-quality:
  stage: analysis
  image: maven:3.9-eclipse-temurin-17
  script:
    - cd healthguardian
    - echo "Running code quality checks with Checkstyle..."
    - mvn checkstyle:checkstyle
  artifacts:
    paths:
      - target/checkstyle-result.xml
  allow_failure: true
  only:
    - main
    - merge_requests

sonarqube-check:
  stage: analysis
  image: maven:3.9-eclipse-temurin-17
  variables:
    # Utilisation de votre IP locale (Wi-Fi) pour être accessible par le Runner
    SONAR_HOST_URL: "http://192.168.1.198:9000"
    SONAR_TOKEN: "sqp_e3b604e497b3f2259531302eae2b49ca192a4218"
    SONAR_PROJECT_KEY: "healthguardian-backend"
  script:
    - cd healthguardian
    - echo "Running SonarQube analysis..."
    - mvn verify sonar:sonar -Dsonar.projectKey=$SONAR_PROJECT_KEY -Dsonar.host.url=$SONAR_HOST_URL -Dsonar.login=$SONAR_TOKE
  allow_failure: true
  only:
    - main
    - merge_requests
```

```
# ================================================================
# STAGE 3: DEPLOY
# ================================================================

docker-deploy:
  stage: deploy
  image: docker:24.0.5
  services:
    - docker:24.0.5-dind
  variables:
    DOCKER_TLS_CERTDIR: "/certs"
  before_script:
    - cd healthguardian
    - echo "$DOCKER_HUB_PASSWORD" | docker login -u "$DOCKER_HUB_USER" --password-stdin
  script:
    - docker build -t $DOCKER_HUB_USER/healthguardian-backend:latest .
    - docker push $DOCKER_HUB_USER/healthguardian-backend:latest
  only:
    - main
```

4.2 Fichiers .gitlab-ci.yml

4.2.1 Fichier .gitlab-ci.yml / FrontEnd

.gitlab-ci.yml 440 B

```
stages:
  - deploy

deploy_to_docker_hub:
  stage: deploy
  image: docker:24.0.5
  services:
    - docker:24.0.5-dind
  variables:
    DOCKER_TLS_CERTDIR: "/certs"
  before_script:
    - echo "$DOCKER_HUB_PASSWORD" | docker login -u "$DOCKER_HUB_USER" --password-stdin
  script:
    - docker build -t "$DOCKER_HUB_USER/healthguardian-frontend:latest" .
    - docker push "$DOCKER_HUB_USER/healthguardian-frontend:latest"
  only:
    - main

```

4.2.2 Fichier .gitlab-ci.yml / AI

.gitlab-ci.yml 487 B

```
image: docker:latest

services:
  - docker:dind

stages:
  - build-and-deploy

deploy-to-dockerhub:
  stage: build-and-deploy
  script:
    - echo "Connexion à Docker Hub..."
    - docker login -u "$DOCKER_HUB_USER" -p "$DOCKER_HUB_PASSWORD"
    - echo "Construction de l'image Docker..."
    - docker build -t "$DOCKER_HUB_USER/healthguardian-ai:latest" .
    - echo "Push de l'image vers Docker Hub..."
    - docker push "$DOCKER_HUB_USER/healthguardian-ai:latest"
  only:
    - main

```

4.2.3 Fichier .gitlab-ci.yml / Deploy

.gitlab-ci.yml 1.69 KiB

```yaml
image: docker:24.0.5

services:
  - docker:24.0.5-dind

variables:
  # Permet à Docker-in-Docker de communiquer
  DOCKER_HOST: tcp://docker:2375
  DOCKER_TLS_CERTDIR: ""
  # Configuration pour que les tests puissent attendre assez longtemps le démarrage des services
  DOCKER_DRIVER: overlay2

stages:
  - test-deploy

test-docker-compose:
  stage: test-deploy
  before_script:
    # Installation de docker-compose et curl
    - apk add --no-cache docker-compose curl
  script:
    - echo "🚀 Starting HealthGuardian environment..."

    # 1. Lancer les conteneurs en arrière-plan
    - docker-compose up -d

    # 2. Attendre que la base de données soit prête (MySQL prend du temps)
    - echo "⏳ Waiting for Database and Services to start (60s)..."
    - sleep 60

    # 3. Vérifier l'état des conteneurs
    - docker-compose ps

    # 4. Vérifier que les 5 conteneurs sont bien démarrés
    - if [ $(docker-compose ps -q | wc -l) -lt 5 ]; then echo "❌ Some containers failed to start"; docker-compose logs; exit 1; fi

    # 5. TEST: Vérifier que le Backend répond
    # Dans Gitlab CI (Dind), les services sont accessibles via 'docker' ou 'localhost' selon la config
    - echo "🔍 Testing Backend connectivity..."
    - curl --retry 5 --retry-delay 5 --retry-connrefused -v http://docker:8081/actuator/health || echo "⚠️ Backend check warning (might need m

    # 6. TEST: Vérifier que l'IA répond
    - echo "🔍 Testing AI Service connectivity..."
    - curl --retry 5 --retry-delay 5 --retry-connrefused -v http://docker:8000/docs || echo "⚠️ AI check warning"

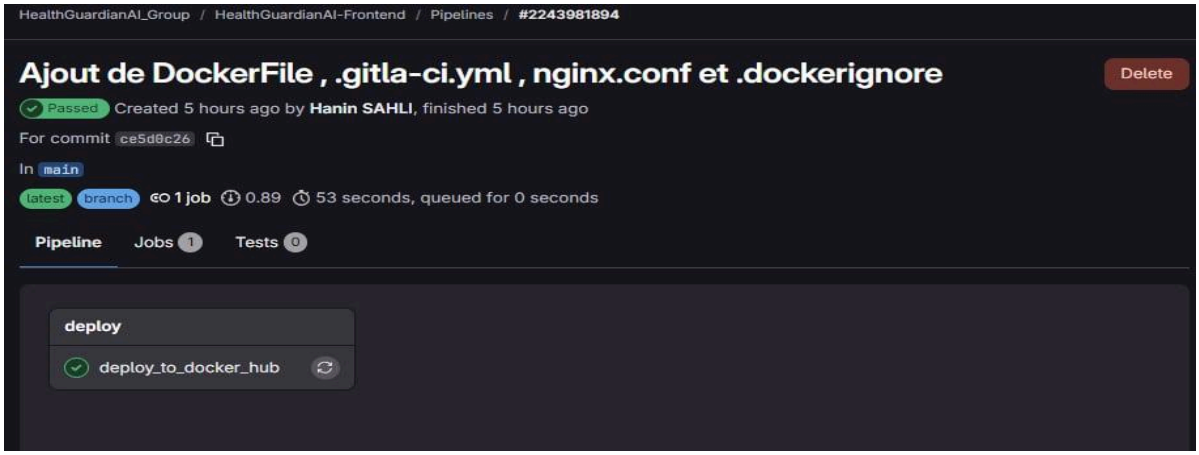
  after_script:
    # Nettoyage
    - docker-compose down
  only:
    - main
```

Chapitre 5

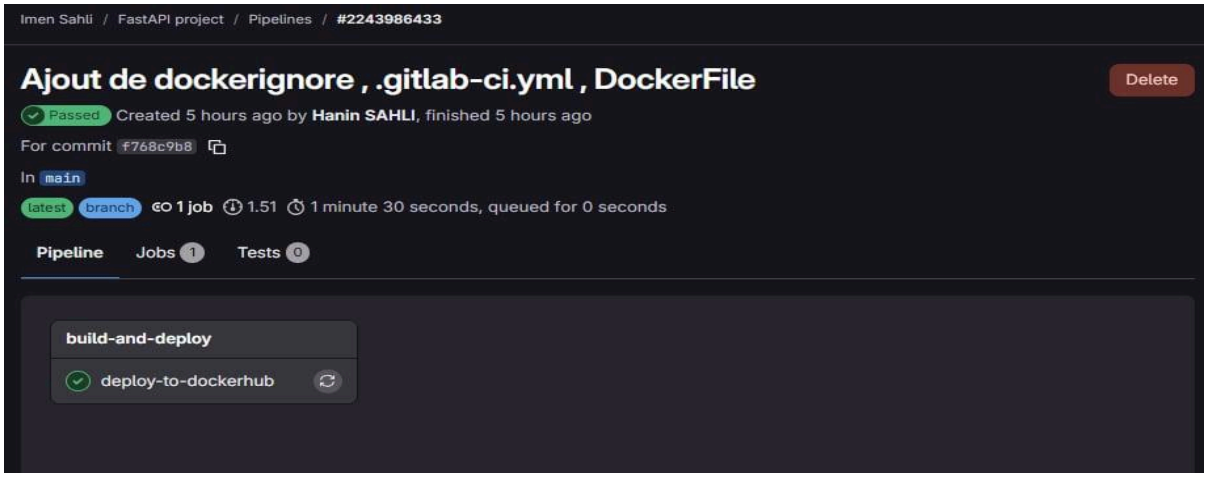
exécution de pipeline ci/cd

5.1 Vue ensemble des pipelines

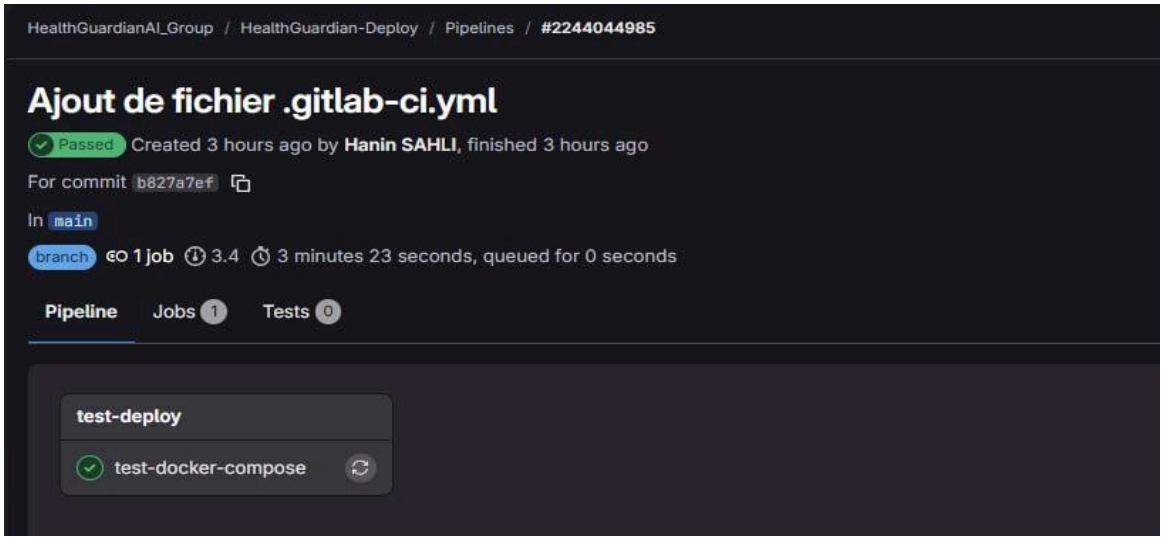
5.1.1 pipeline partie frontEnd



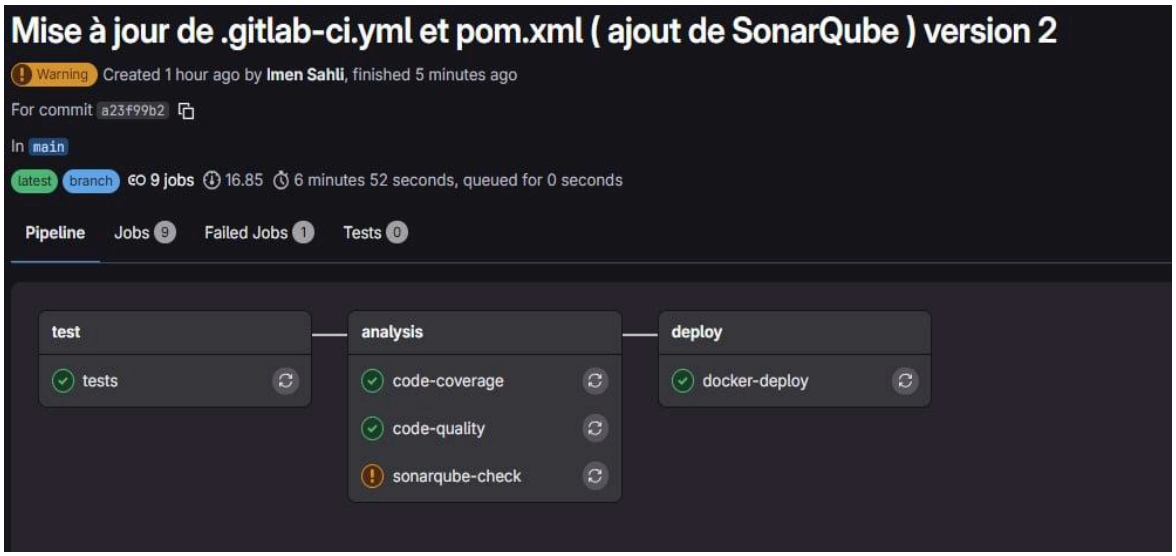
5.1.2 pipeline partie ai



5.1.3 pipeline deploy



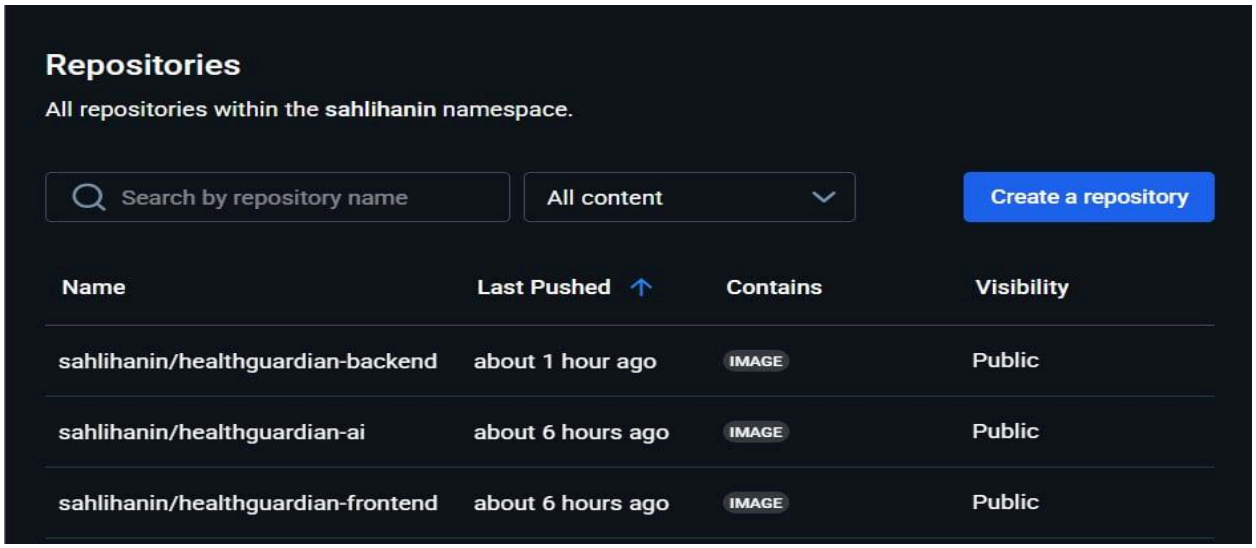
5.1.4 pipeline Backend



Chapitre 6

Deploiment et validation

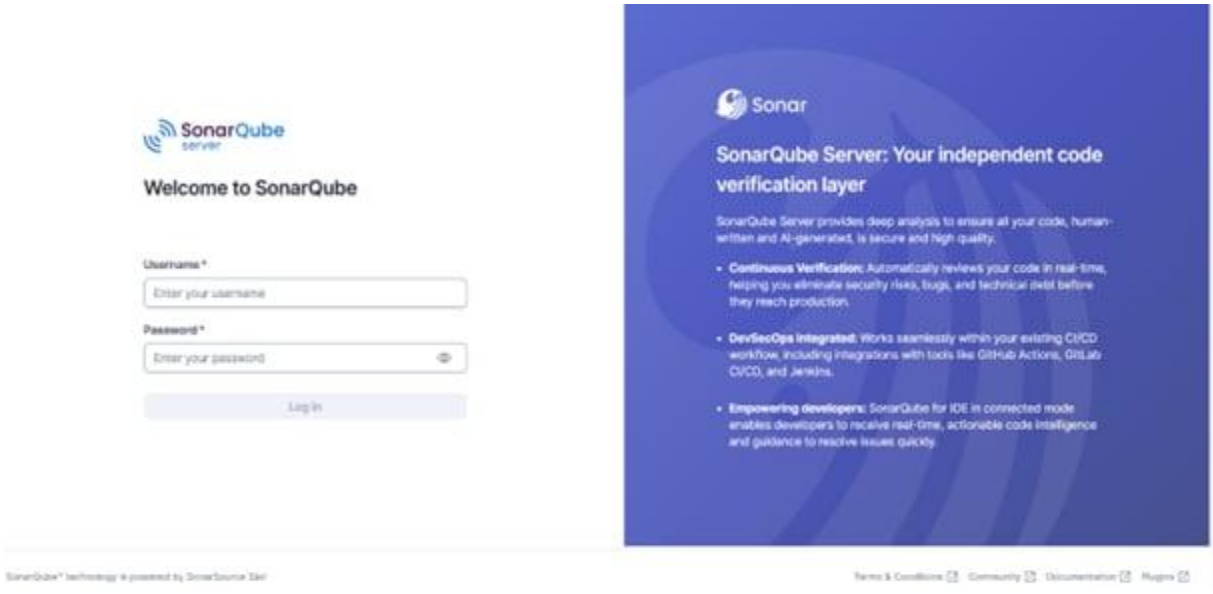
6.1 Images publiées sur DockerHub

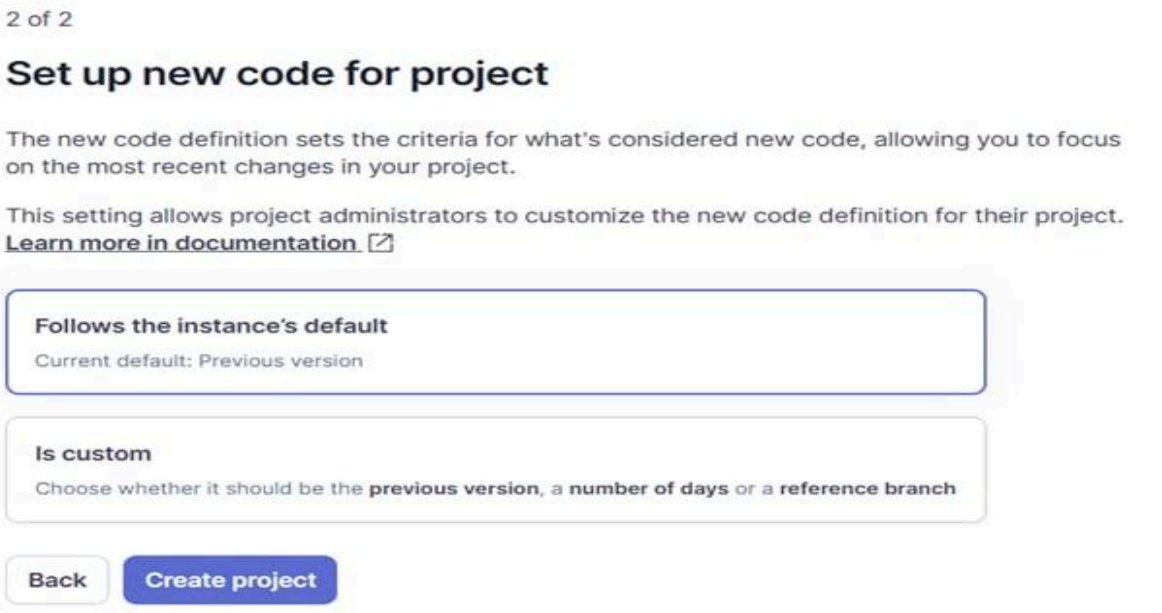


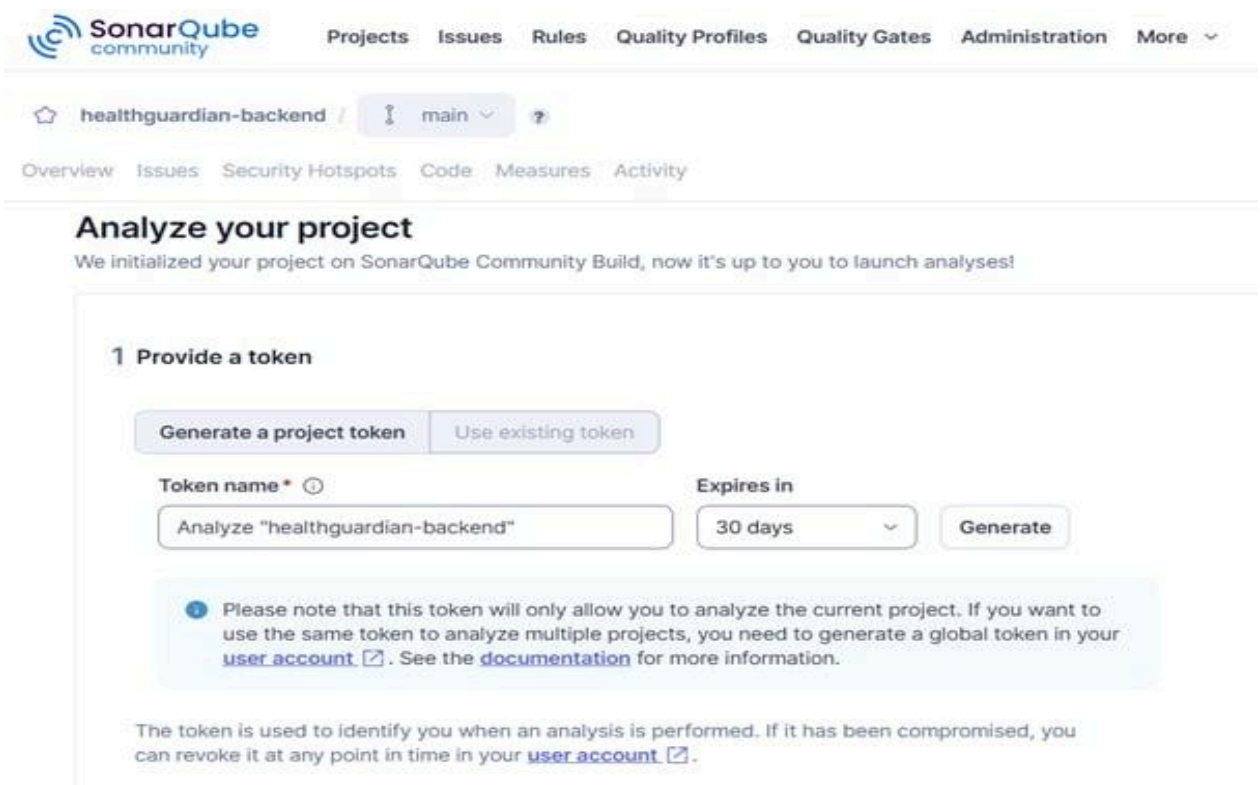


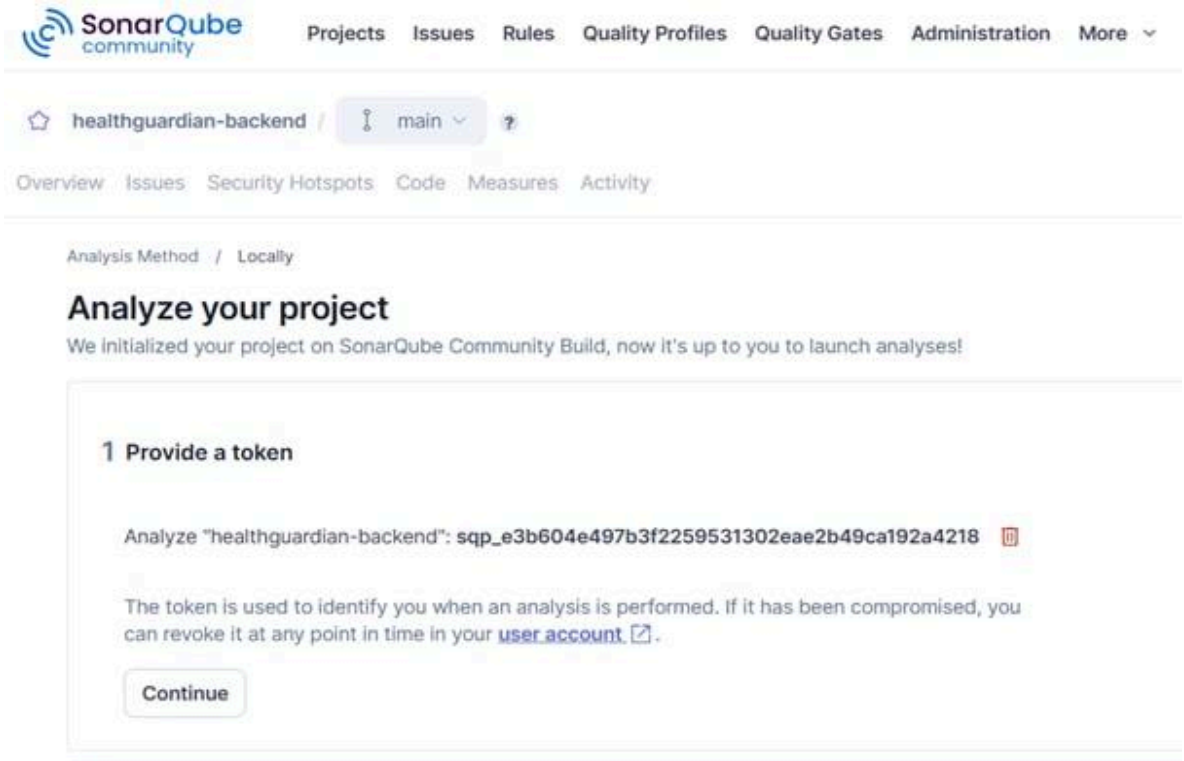
Chapitre 7

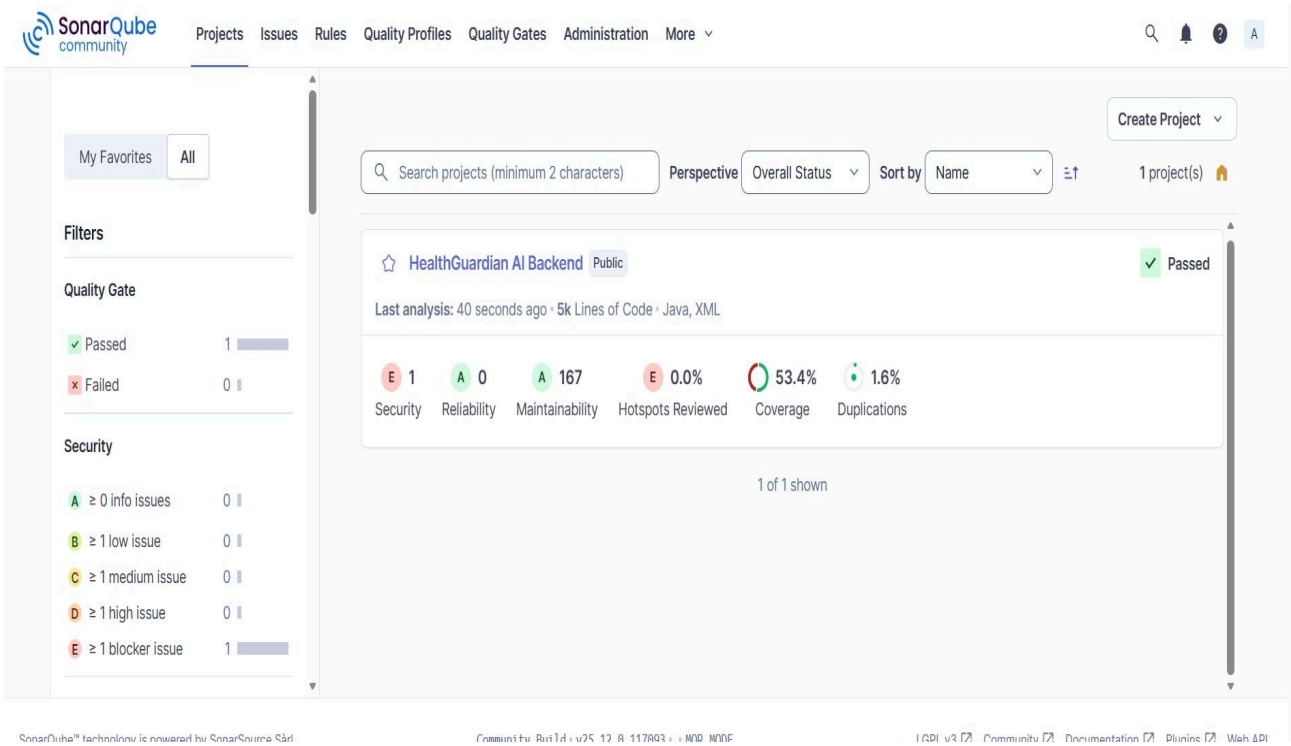
SonarQube






SonarQube community
Projects
Issues
Rules
Quality Profiles
Quality Gates
Administration
More
healthguardian-backend
main
Overview
Issues
Security Hotspots
Code
Measures
Activity
Analyze your project
We initialized your project on SonarQube Community Build, now it's up to you to launch analyses!
1 Provide a token
Generate a project token
Use existing token
Token name *
Analyze "healthguardian-backend"
Expires in
30 days
Generate
Please note that this token will only allow you to analyze the current project. If you want to use the same token to analyze multiple projects, you need to generate a global token in your user account. See the documentation for more information.
The token is used to identify you when an analysis is performed. If it has been compromised, you can revoke it at any point in time in your user account.


SonarQube community
Projects
Issues
Rules
Quality Profiles
Quality Gates
Administration
More
healthguardian-backend
main
Overview
Issues
Security Hotspots
Code
Measures
Activity
Analysis Method / Locally
Analyze your project
We initialized your project on SonarQube Community Build, now it's up to you to launch analyses!
1 Provide a token
Analyze "healthguardian-backend": sqp_e3b604e497b3f2259531302eae2b49ca192a4218
The token is used to identify you when an analysis is performed. If it has been compromised, you can revoke it at any point in time in your user account.
Continue


SonarQube community
Projects
Issues
Rules
Quality Profiles
Quality Gates
Administration
More
A
My Favorites
All
Filters
Quality Gate
Passed 1
Failed 0
Security
A ≥ 0 info issues 0
B ≥ 1 low issue 0
C ≥ 1 medium issue 0
D ≥ 1 high issue 0
E ≥ 1 blocker issue 1
Create Project
Search projects (minimum 2 characters)
Perspective
Overall Status
Sort by
Name
1 project(s)
HealthGuardian AI Backend
Public
Passed
Last analysis: 40 seconds ago • 5k Lines of Code • Java, XML
E 1
Security
A 0
Reliability
A 167
Maintainability
E 0.0%
Hotspots Reviewed
53.4%
Coverage
1.6%
Duplications
1 of 1 shown
SonarQube™ technology is powered by SonarSource Sàrl
Community Build • v25.12.0.117093 • MQR MODE
LGPL v3
Community
Documentation
Plugins
Web API

Chapitre 8

Conclusion

7.1 Processus réalisé dans l'ordre chronologique

1. Tests locaux : Exécution et validation des tests unitaires et d'intégration

2. Analyse qualité locale : Génération du rapport JaCoCo pour mesurer la couver

ture de code

3. Dockerisation : Création et optimisation des Dockerfiles

4. Build local : Test avec Docker Compose pour valider le fonctionnement

5. Configuration CI/CD : Mise en place du pipeline GitLab

6. Automatisation : Pipeline exécuté automatiquement à chaque modification

7. Déploiement : Publication automatique sur Docker Hub

8. Validation : Tests de récupération et vérification finale

7.2 Résultats obtenus

— Pipeline CI/CD complet : 5 stages automatisés (Build, Test, Quality, Package,

Deploy)

— Testsvalidés:Tests unitaires et d'intégration exécutés localement puis automatisés

— Qualité assurée : Rapports JaCoCo générés localement et dans le pipeline

— Images Docker optimisées : 3 images avec multi-stage builds

— Déploiement continu : Publication automatique sur Docker Hub

— Validation réussie : Application fonctionnelle après déploiement